route, we can use the “straight-line distance to the customer’s address” as the heuristic: this will guide search, favoring paths that gets closer to the goal over those that are directed away from it; in turn, it will require to visit less vertices before reaching the goal.

Figure 14.22 shows how A* would find the best route for our previous example, the one to which we applied Dijkstra in figure 14.18. While running Dijkstra’s algorithm we would have had to visit all vertices whose distance from source is smaller than 1356 (the total distance of the shortest path from source to goal), A* can reach the goal faster and, although it still visits some vertices not in the shortest path, it doesn’t go through vertices v_A, v_B, v_C, v_F, v_N, v_R, which are instead visited by Dijkstra’s.

In this particular example, with A* we can save traversing 6 edges over 23, which is quite good (a 25% save), especially considering that there are two paths with a very close weight, differing by just 2 meters.

Considering road distance as edges’ weight and straight-line distance as the heuristic, we can see that straight-line distance is certainly optimistic, because the road distance can never be shorter, at most it can be the same.

Straight-line distance is also a consistent heuristic; in fact, if we apply our choices to the condition for consistency, we get:

```
straight_line(u, goal) ≤ road_distance(v, u) +
straight_distance(v)
```